GENERALITAT VALENCIANA

ies SERPIS | departament informàtica

DESARROLLO APLICACIONES WEB EN ENTORNO CLIENTE

CURSO 26/ 27

2º DAW

UT 7.4. Callbacks, promesas y async/await en JavaScript

Rafael Torres Ferrer

UNIDAD 07.4

Programación asíncrona

Callbacks, promesas y async/await en JavaScript

```
const datos = await cargar();
console.log('¡Listo!');
```

¿Qué vamos a ver hoy?

1

Síncrono y asíncrono

Por qué JavaScript no espera

25 min

2

Callback hell

El problema de anidar callbacks

10 min

3

Promesas

Crear, then, catch y Promise.all

40 min

4

async / await

Código asíncrono que se lee de arriba abajo

20 min

5

Ejercicios

Ejercicios de menor a mayor dificultad

25 min

¿En qué orden aparecen las letras?

```
console.log('A');
setTimeout(() => console.log('B'), 0);
Promise.resolve().then(() => console.log('C'));
console.log('D');
```

Antes de ejecutar

Escribe tu respuesta en un papel.

Pista: el tiempo de espera es 0.

? ? ? ?

La respuesta: A, D, C, B

A código síncrono

D código síncrono

C promesa

B temporizador

1. Primero se ejecuta todo el código síncrono, línea a línea.
2. Después se atienden las promesas que ya están cumplidas.
3. Por último se atienden los temporizadores, aunque sean de 0 ms.

«0 milisegundos» no significa «ahora», sino «cuando termines lo que estás haciendo».

1

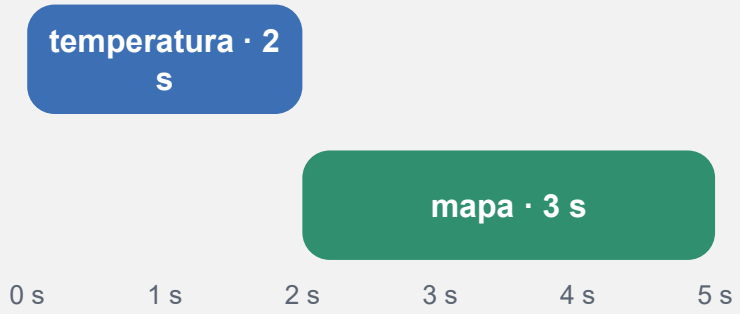
Síncrono y asíncrono

¿Por qué JavaScript no espera?

Dos servicios, dos formas de cargar

Síncrono

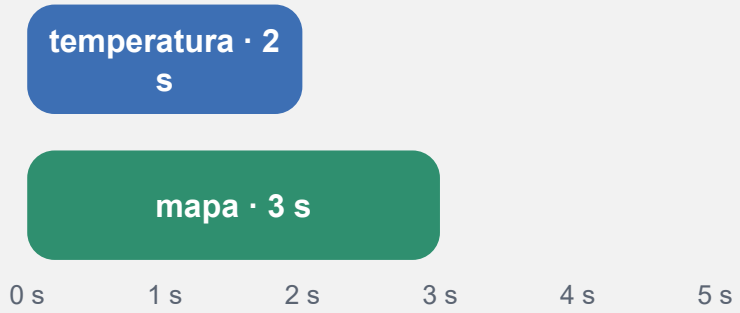
5 s



El mapa no empieza hasta que llega la temperatura.

Asíncrono

3 s



Cada carga va por su cuenta. El usuario espera menos.

Un solo hilo que no se queda esperando

JavaScript ejecuta el código instrucción a instrucción, pero delega las esperas: red, temporizadores y otras operaciones de entrada/salida.

1 Toma el pedido

El camarero atiende una mesa cada vez.

Tu código se ejecuta línea a línea.

2 Lo pasa a cocina

No se queda mirando el fogón: sigue atendiendo mesas.

La petición se lanza y el programa continúa.

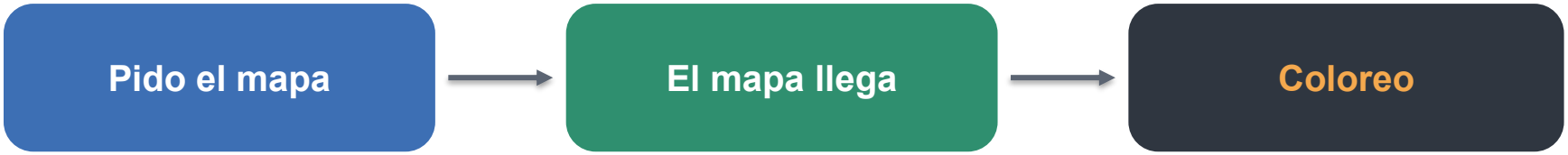
3 Cocina avisa

Cuando el plato está listo, lo sirve.

Llega el resultado y se ejecuta el callback.

El problema nuevo: sincronizar

Tengo el código para colorear el mapa, pero ¿ha llegado ya el mapa?



Solución conocida: una función callback que se ejecuta cuando el mapa ha cargado.

```
componente.haCargado(() => { colorear() });
```

2

Callback hell

Cuando hay que encadenar varias tareas

Cargar, colorear y animar con callbacks

```
boton.addEventListener('click', function (ev) {
  cargarMapa(componente, function (error) {
    if (error) {
      console.log('Error al cargar el mapa');
    } else {
      colorear(componente, function (error) {
        if (error) {
          console.log('Error al colorear');
        } else {
          animar(componente);
        }
      });
    }
  });
});
```

Se desplaza a la derecha

Cada paso añade un nivel más de sangría.

Los errores se repiten

Hay que comprobar el error en cada nivel.

Funciona, pero no se lee

Añadir o cambiar un paso es arriesgado.

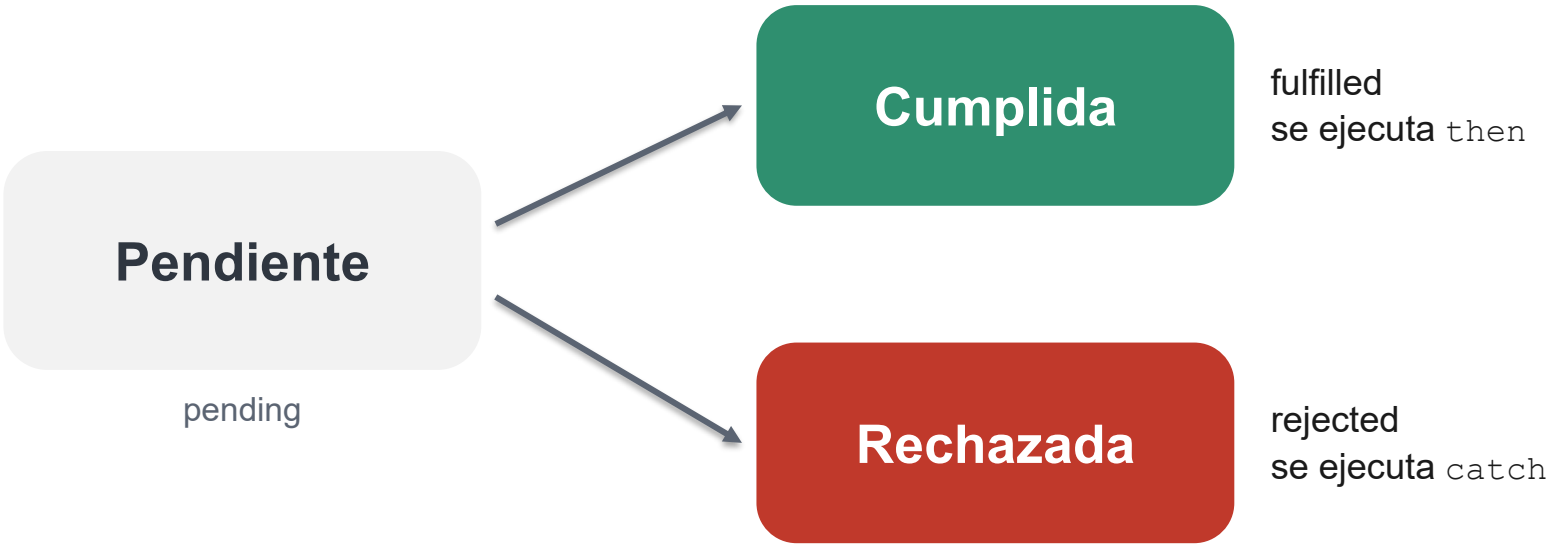
3

Promesas

Un objeto que representa un resultado futuro

Una promesa tiene tres estados

Una promesa es un objeto que representa el resultado de una tarea asíncrona que todavía no ha terminado.



Finalizada (settled): ya no está pendiente. Una promesa finalizada no vuelve a cambiar de estado.

Crear una promesa

```
let promesa = new Promise((resolver, rechazar) => {
  // tarea asíncrona
  if (/* ha ido bien */) {
    resolver(resultado);
  } else {
    rechazar(Error('Mensaje de error'));
  }
});
```

resolver

La tarea ha terminado bien. Envía el resultado.

rechazar

La tarea ha fallado. Envía un objeto de error.

La tarea empieza en el momento de crear la promesa

Recoger el resultado: then y catch

```
promesa
  .then((resultado) => {
    // si la promesa se cumple
  })
  .catch((error) => {
    // si la promesa se rechaza
  });
```

¿Quién crea la promesa?

Decide cuándo llamar a `resolver` o a `rechazar`.

¿Quién la consume?

Indica qué hacer en cada caso con `then` y `catch`.

`then` y `catch` devuelven una **promesa nueva**. Por eso se pueden escribir uno detrás de otro.

Un primer ejemplo: ¿son iguales?

```
var promesa = new Promise((resolver, rechazar) => {
  let n1 = 2;
  let n2 = 2;
  if (n1 == n2) resolver('¡Son iguales!');
  else rechazar(Error('Algo raro ha pasado'));
});

promesa
  .then((respuesta) => {
    console.log(respuesta);
  })
  .catch((error) => {
    console.log(error.message);
  });
```

Con n1 = 2 y n2 = 2

```
¡Son iguales!
```

Se llama a `resolver` y se ejecuta `then`.

Con n1 = 2 y n2 = 3

```
Algo raro ha pasado
```

Se llama a `rechazar` y se ejecuta `catch`.

Encadenar promesas

```
cargarMapa()
  .then((mapa) => cargarPlantilla(mapa))
  .then((mapaConPlantilla) => colorear(mapaConPlantilla))
  .catch((error) => console.log('Error en la carga:', error));
```

1

Pasos en orden

Un `then` no se ejecuta hasta que termina la promesa anterior.

2

El valor pasa

Cada `then` recibe lo que devolvió el `then` anterior.

3

Un solo catch

Si falla cualquier paso, se saltan los demás y se va al `catch`.

El error más habitual: olvidar el return

Sin return

```
cargarMapa()
  .then((mapa) => {
    colorear(mapa);
  })
  .then((mapa) => {
    animar(mapa); // mapa es undefined
  });
```

El segundo `then` no espera a `colorear` y recibe `undefined`.

Con return

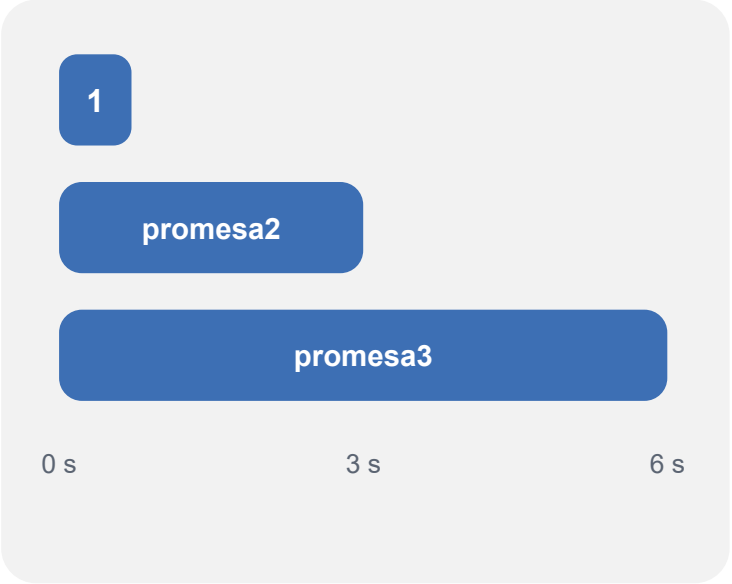
```
cargarMapa()
  .then((mapa) => {
    return colorear(mapa);
  })
  .then((mapa) => {
    return animar(mapa);
  });
```

Cada paso espera al anterior y recibe su resultado.

Promise.all: esperar a todas

```
let conjunta = Promise.all([
  promesa1,    // al instante
  promesa2,    // 3 segundos
  promesa3,    // 6 segundos
]);

conjunta.then((resultados) => { ... });
```



Se cumple cuando se cumplen **todas**: entrega los resultados en el mismo orden.

Se rechaza en cuanto falla **una**, sin esperar a las demás.

Otros métodos de Promise

`Promise.resolve(v)`

Crea una promesa ya cumplida con el valor indicado.

`Promise.reject(e)`

Crea una promesa ya rechazada con el error indicado.

`Promise.allSettled([...])`

Espera a que todas terminen, bien o mal. Nunca se rechaza.

`Promise.race([...])`

Termina igual que la primera promesa que finalice.

4

async / await

Código asíncrono que se lee de arriba abajo

Funciones async y operador await

```
async function esperarTiempos() {
  let mensaje1 = await promesa1;
  console.log(mensaje1);
  let mensaje2 = await promesa2;
  console.log(mensaje2);
}

esperarTiempos();
```

1. Una función `async` siempre devuelve una promesa.

2. `await` solo se usa dentro de una función `async`.

Pendiente la función se pausa; el resto del programa sigue.

Cumplida `await` devuelve el resultado y se ejecuta la línea siguiente.

Rechazada `await` lanza el error.

¿6 segundos o 9 segundos?

Promesas creadas antes

```
const p2 = esperar(3000);
const p3 = esperar(6000);
await p2;
await p3;
```



Promesas creadas tras cada await

```
await esperar(3000);
await esperar(6000);
```



La tarea empieza al crear la promesa, no al hacer await

Gestionar errores con try..catch

```
async function falla() {
  try {
    let resultado = await Promise.reject(
      Error('Promesa rechazada')
    );
  } catch (error) {
    console.log(error.message);
  }
}

falla(); // Escribe: Promesa rechazada
```

await lanza el error

Una promesa rechazada se comporta como un `throw`.

Un solo bloque

Un `try..catch` cubre todos los `await` que contiene.

También vale catch

Se puede seguir usando `.catch()` sobre la promesa.

El mismo proceso, escrito de tres formas

Callbacks

```
cargarMapa((err, mapa) => {
  if (err) return fallo(err);
  colorear(mapa, (err, mapa) => {
    if (err) return fallo(err);
    animar(mapa, (err) => {
      if (err) return fallo(err);
      console.log('Hecho');
    });
  });
});
```

Promesas

```
cargarMapa()
  .then((m) => colorear(m))
  .then((m) => animar(m))
  .then(() => {
    console.log('Hecho');
  })
  .catch((e) => fallo(e));
```

async / await

```
try {
  let m = await cargarMapa();
  m = await colorear(m);
  m = await animar(m);
  console.log('Hecho');
} catch (e) {
  fallo(e);
}
```

5

Ejercicios

Nueve ejercicios de menor a mayor dificultad

A trabajar

1. Predecir la salida — Básico
2. Predecir la salida con await — Básico
3. Crear una promesa: esperar — Básico
4. Resolver y rechazar: el dado — Medio
5. Encontrar los errores — Medio
6. Salir del callback hell — Medio
7. El semáforo — Medio
8. Secuencial o en paralelo — Avanzado
9. Tiempo límite — Avanzado

Trabajo en casa

Tres ideas

1. JavaScript tiene un solo hilo, pero no se queda esperando.
2. Una promesa representa un resultado futuro: se cumple o se rechaza, una sola vez.
3. `async`/`await` es la forma más legible de trabajar con promesas.

Tres preguntas

? ¿Cuántas veces puede cambiar de estado una promesa?

? ¿Qué recibe un `then` si el anterior no hace `return`?

? ¿Qué devuelve siempre una función `async`?